

Advanced NLP – Exercise 2

Part 1

Netanel Azran

June 2026

Question 1 — Societal Bias Beyond Gender

Paper: Moin Nadeem, Anna Bethke, and Siva Reddy. “StereoSet: Measuring stereotypical bias in pretrained language models.” *ACL-IJCNLP 2021*.

StereoSet is a crowdsourced benchmark for measuring how much pretrained language models rely on social stereotypes. It covers four bias categories — race, religion, profession, and gender — with roughly 17,000 items in total. Each item presents a short context followed by three candidate continuations: a *stereotypical* one, an *anti-stereotypical* one, and a semantically *unrelated* one. For example, given “*Arabs are...*,” the options might be “violent and dangerous,” “peaceful and welcoming,” and “made of bricks.”

The paper proposes two metrics. The *Stereotype Score* (SS) measures how often the model prefers the stereotypical continuation over the anti-stereotypical one (50 = unbiased). The *Language Model Score* (LMS) checks that the model still prefers meaningful text over nonsense, serving as a sanity check. The two are combined into the *ICAT score*, which rewards models that are both capable and minimally biased. Tested models (BERT, RoBERTa, XLNet, GPT-2) all show clear stereotypical preferences in the race and religion subsets.

Question 2 — Three Efficiency Methods

Method 1: Quantization (Inference)

Quantization reduces the numerical precision of model weights — from float32 down to float16, INT8, or INT4. This can be applied after training (*post-training quantization*) without any retraining, making it very cheap to deploy.

The main saving is *memory*: a 7B-parameter model shrinks from ~28 GB in float32 to ~3.5 GB in INT4, which is the difference between needing a data-centre GPU and fitting on a consumer card. Inference speed also improves because lower-precision arithmetic is faster on modern hardware.

The trade-off is a small drop in accuracy, which becomes more noticeable at very aggressive quantization (4-bit). Quantization-aware training recovers most of the loss but requires an extra training phase.

Method 2: LoRA (Training)

LoRA (Hu et al., 2022) fine-tunes a model by freezing all pretrained weights and injecting small low-rank adapter matrices into each transformer layer. Only those adapters are updated; the rest of the model is untouched. After training, the adapters can be merged back into the base weights, so inference cost does not change.

For a 7B model, the number of trainable parameters drops from 7 billion to roughly 20–30 million (under 0.5%), making it practical to fine-tune large models on a single consumer GPU.

The trade-off is expressiveness: because weight updates are constrained to a low-rank subspace, LoRA may fall slightly behind full fine-tuning on some tasks, though in practice the gap is small.

Method 3: Knowledge Distillation (Modeling)

Knowledge distillation (Hinton et al., 2015) trains a small *student* model to mimic a large *teacher* model. Instead of training on hard one-hot labels, the student matches the teacher’s full output distribution (“soft” probabilities), which carries richer information about similarity between classes. A well-known NLP example is DistilBERT — a 66M-parameter student trained from BERT-base (110M parameters) that retains $\sim 97\%$ of BERT’s GLUE performance while being 40% smaller and 60% faster.

The trade-off is extra training cost: generating soft labels requires running the large teacher, so distillation takes more compute than training the student from scratch.

Pairwise Combinations

Quantization + LoRA. These two are already combined in practice as **QLoRA** (Dettmers et al., 2023). The frozen base model is quantized to 4-bit (saving the memory needed to hold it), while the LoRA adapters are trained in 16-bit. This makes it possible to fine-tune a 65B model on a single 48 GB GPU — something neither method achieves alone.

Quantization + Knowledge Distillation. A natural pipeline: first distill the large teacher into a smaller student, then quantize the student for deployment. The two savings stack — a student that is $4\times$ smaller than the teacher, then quantized to INT8, ends up using about one-eighth the memory of the original. The only cost is two sequential training phases.

LoRA + Knowledge Distillation. These methods target different problems — LoRA reduces fine-tuning cost while distillation reduces model size — so they are complementary rather than redundant. One practical combination is to distill a large model into a smaller student, then use LoRA to adapt that student cheaply to a downstream task. Useful when both deployment efficiency and training efficiency are required at the same time.

Advanced NLP – Exercise 2

Part 2: Sentiment Analysis Failure Analysis

Code: <https://github.com/netanelazran11/ANLP-2026/tree/main/ex2>

Models used: Primary — Qwen/Qwen3-8B; Second — meta-llama/Llama-3.1-8B-Instruct.

Break the Model

Failure type: implicit negative sentiment. All nine inputs are genuinely negative in meaning, but none contains an explicit negative word. The negativity must be inferred from understatement or implied real-world consequence — e.g., an umbrella lasting ten seconds in a storm, or a battery that barely reaches the next room. A model that classifies by surface vocabulary has no negative signal to latch on to and defaults to **Positive**.

Table 1: Break examples — Qwen/Qwen3-8B (primary model). Bold = wrong prediction.

#	Input	Gold	Model output
1	This umbrella kept me perfectly dry for the first ten seconds of the storm.	Neg	Pos
2	The battery lasts just long enough to get from my bedroom to the kitchen.	Neg	Pos
3	Customer support got back to me in only three hours, which was faster than my last ticket.	Neg	Pos
4	The phone survived a drop from pocket height, as long as I was sitting down at the time.	Neg	Pos
5	After only two visits to the mechanic this month, the car is running perfectly again.	Neg	Pos
6	The noise-cancelling headphones reduced the drilling noise next door to a very manageable level.	Neg	Pos
7	I have now successfully assembled three of the seven required pieces.	Neg	Pos
8	The waterproofing performed admirably in light drizzle, the only condition I had the chance to test.	Neg	Pos
9	I got to sample the entire menu, as each dish came back to the kitchen at least once.	Neg	Pos
Failures (non-Negative predictions)			8/9

Why is the Model Wrong?

Hypothesis: The model relies on explicit sentiment vocabulary as a shortcut. When the surface words are neutral or positive (“perfectly dry”, “successfully assembled”, “admirably”), it predicts **Positive**, ignoring the pragmatic implication that the stated performance is far below any reasonable expectation.

To test this, I designed eight controlled inputs that vary *only* the degree of lexical explicitness, keeping the underlying scenario fixed (a failing umbrella). H1–H3 contain explicit negative vocabulary (model should be correct); H4–H6 use pure understatement with no negative words (predicted failures); H7–H8 are positive controls.

Table 2: Hypothesis tests — Qwen/Qwen3-8B. Bold = wrong prediction.

#	Input	Gold	Model output
H1	This umbrella broke in the first minute of rain and I got completely soaked.	Neg	Neg
H2	This umbrella is useless. It fell apart immediately and left me drenched.	Neg	Neg
H3	Oh great, another umbrella that lasts all of ten seconds in the rain.	Neg	Neg
H4	This umbrella kept me dry for the first ten seconds of the storm.	Neg	Pos
H5	The battery lasts just long enough to reach the coffee machine from my desk.	Neg	Neg
H6	I used this umbrella once before going back to my old one.	Neg	Neg
H7	This umbrella is fantastic. It kept me completely dry through two hours of heavy rain.	Pos	Pos
H8	This umbrella handled a two-hour downpour without a single leak.	Pos	Pos

The model correctly classifies all explicit negatives (H1–H3) and both positive controls (H7–H8), confirming it has no general difficulty with sentiment classification. It fails on H4 (pure understatement) as predicted, while H5–H6 are classified correctly, suggesting the model can occasionally pick up on very specific cues (an absurdly short time window, or the phrase “went back to my old one”). Overall the results support the hypothesis: *the more implicit the negativity, the more likely the model is to fail*.

Try Another Model

I ran the same nine break examples through **meta-llama/Llama-3.1-8B-Instruct**.

Llama-3.1-8B fails on 6 out of 9 examples, confirming the failure type is not specific to Qwen. One difference is that Llama sometimes outputs **Neutral** (items 4 and 7) rather than **Positive**, suggesting it is somewhat less certain when no explicit vocabulary is present, but it still cannot infer the negative meaning from understatement alone.

Proposed Solution

Two-step prompting. The root cause is that the model never performs the pragmatic inference step — it goes straight to classification without reasoning about what the sentence implies. The proposed fix is to make that step explicit:

1. Ask the model to describe what the sentence implies about the product: “*In one sentence, what does the following review imply about the product’s quality? [input]*”
2. Classify the generated paraphrase, not the original sentence.

For example, given “*This umbrella kept me perfectly dry for the first ten seconds of the storm,*” step 1 would produce something like “*The umbrella provides almost no protection from rain,*”

Table 3: Break examples — meta-llama/Llama-3.1-8B-Instruct. Bold = wrong prediction.

#	Input	Gold	Model output
1	This umbrella kept me perfectly dry for the first ten seconds of the storm.	Neg	Neg
2	The battery lasts just long enough to get from my bedroom to the kitchen.	Neg	Neg
3	Customer support got back to me in only three hours, which was faster than my last ticket.	Neg	Pos
4	The phone survived a drop from pocket height, as long as I was sitting down at the time.	Neg	Neu
5	After only two visits to the mechanic this month, the car is running perfectly again.	Neg	Pos
6	The noise-cancelling headphones reduced the drilling noise next door to a very manageable level.	Neg	Pos
7	I have now successfully assembled three of the seven required pieces.	Neg	Neu
8	The waterproofing performed admirably in light drizzle, the only condition I had the chance to test.	Neg	Pos
9	I got to sample the entire menu, as each dish came back to the kitchen at least once.	Neg	Neg
Failures (non-Negative predictions)			6/9

which the standard classifier correctly labels **Negative**. This requires no additional training data and adds only one forward pass per input.